

Malware Analysis: javaw.exe - Rust-Based InfoStealer with Process Injection and Sophisticated Anti-Analysis

ID: vuln-0001

Severity: CRITICAL

Found: 2026-05-19 22:28:46 UTC

Target: /workspace/malware-analysis/javaw.exe

CWE: CWE-200

CVSS: 9.6

Description

Comprehensive static malware analysis of the file 'javaw.exe' (22.4 MB PE32+ executable). This file is a highly sophisticated Rust-based information stealer (InfoStealer) with advanced process injection, anti-sandbox, anti-VM, anti-debugging capabilities, and encrypted command-and-control communication.

Key Classification: Rust-compiled Windows InfoStealer with hybrid encryption, direct NT syscall-based process injection, browser credential theft targeting Chrome/Firefox/Edge, and comprehensive sandbox/VM avoidance.

Language: Rust, compiled with rustc (commit 59807616e1fa2540724bfbac14d7976d7e4a3860) on 2026-05-18

Architecture: 64-bit Windows PE32+ GUI application (IMAGE_SUBSYSTEM_WINDOWS_GUI - no console window)

File size: 22,399,488 bytes (21.36 MB) - 98.6% of the binary consists of a single large .rdata payload section

File Hashes:

- MD5: 1c429a60c823099b4dbb2b7be93ac10b
- SHA1: b722632464f062050a2e5f2f469d786a99ce37e6
- SHA256: 6df297c6ee6c5278dbec4c1c553e71dfd4acae9b559012c22b8e836e21dfdfa3

Overall file entropy: 6.06 (moderately high, consistent with compressed/encoded payload sections)

Impact

This malware poses a critical threat to any system on which it is executed:

Data Theft:

- Complete browser credential theft from Chrome, Firefox, and Edge including saved passwords, cookies, autofill data, and browsing history
- Potential crypto wallet data exfiltration
- System and user information harvesting

Process Injection:

- Ability to inject malicious code into legitimate processes using direct NT syscalls, bypassing userland API hooks and EDR/AV monitoring
- Capability to establish persistent code execution within trusted system processes

Encrypted C2 Communication:

- Uses hybrid encryption (AES-GCM + Curve25519/ML-KEM post-quantum) for C2 traffic, making network detection extremely difficult
- Encrypted payload in the .rdata section prevents signature-based detection

Evasion:

- Extensive anti-sandbox/anti-VM checks against VMware, VirtualBox, Hyper-V, Cuckoo, Sandboxie, and analysis environments
- Direct syscall execution bypasses userland hooks
- Application masquerades as javaw.exe (legitimate Java executable name)
- No console window (GUI subsystem) for stealth

Business Impact: Full compromise of browser-stored credentials, session tokens, and sensitive user data. Potential for lateral movement via process injection. Extremely difficult to detect with traditional signature-based AV.

Technical Analysis

Detailed Technical Analysis

1. PE Structure Anomalies

The executable exhibits multiple structural anomalies consistent with malware:

Unusual Section Layout: 11 sections (versus typical 6-8 for legitimate Rust binaries). The entry point resides in the .data section (executable, readable, containing code) rather than the conventional .text section — a strong indicator of intentional obfuscation.

Massive .rdata Payload Section: The second .rdata section (RVA 0x48000) occupies 20.8 MB, representing 98.6% of the total file. This section contains encrypted/encoded payload data, including JPEG images likely used as decoys or configuration resources.

High-Entropy .tls Section: The Thread Local Storage (.tls) section exhibits entropy of 7.53 (near-maximum for 8-bit data), indicating packed/encrypted TLS callback functions. TLS callbacks execute before the entry point, enabling stealthy initialization and anti-debugging measures.

Abnormal Section Size Ratios:

- .CRT section: raw size 4.92x virtual size
- .tls section: raw size 32x virtual size (highly anomalous)

2. Anti-Analysis Capabilities

Anti-Sandbox Mechanism Identification: The binary contains hardcoded strings to detect analysis environments:

- **Process detection:** vmtoolsd.exe (VMware Tools), VBoxService.exe (VirtualBox Guest Additions)
- **Driver detection:** vmhgfs.sys (VMware HGFS), vmbus.sys (Hyper-V), SynthVid.sys (Hyper-V), VBoxMouse.sys (VirtualBox)
- **DLL detection:** sbiedll.dll (Sandboxie), cuckoomon.dll (Cuckoo Sandbox), dbghelp.dll (debugging), api_log.dll, dir_watch.dll, pstorec.dll (Protected Storage), vmcheck.dll (VM detection), wpespy.dll (packet sniffing), cmdvrt32/64.dll (Comodo Sandbox)
- **Path detection:** C:\analysis, C:\sandbox, C:\insidetm, C:\cuckoo, C:\sample
- **Username blacklist:** sandbox, maltest, virus, malware, cuckoo, sample, analyse (checked via GetUserNameW)
- **Computer name blacklist:** analysis, desktop-win-pc, win-pc

Anti-Debug Capabilities:

- AddVectoredExceptionHandler - vectored exception handling for anti-debug and crash prevention
- SetUnhandledExceptionFilter - global exception filter
- RtlCaptureContext - context capture for integrity checking
- QueryPerformanceCounter / GetTickCount64 - timing analysis detection (slow-stepping)

Process/Module Enumeration:

- CreateToolhelp32Snapshot + Module32First/NextW - enumerates running processes and modules to detect analysis tools

3. Process Injection Capabilities

The binary implements direct NT syscall-based process injection, bypassing userland API hooks placed by EDR/AV solutions. Embedded compound string reveals the exact syscall chain:

```
chameleon-aes-v1aes-gcm-keymlkem-placeholder-v1chameleon-hybrid-v1session-key  
NtMapViewOfSection NtUnmapViewOfSection  
NtOpenProcess NtCreateThreadEx NtWriteVirtualMemory  
NtProtectVirtualMemory NtAllocateVirtualMemory  
NtFlushInstructionCache NtQuerySystemInformation NtClose
```

The injection flow is reconstructed as:

1. **Target selection:** NtQuerySystemInformation (enumerate processes) or CreateToolhelp32Snapshot
2. **Target opening:** NtOpenProcess with required access rights
3. **Memory allocation:** NtAllocateVirtualMemory in target process
4. **Code writing:** NtWriteVirtualMemory to inject payload
5. **Memory protection:** NtProtectVirtualMemory to set RX permissions
6. **Section mapping:** NtMapViewOfSection/NtUnmapViewOfSection for shared memory injection
7. **Execution:** NtCreateThreadEx to create remote thread at injected code
8. **Cache sync:** NtFlushInstructionCache

This is a classic "inject-and-execute" pattern commonly used by advanced malware to evade userland monitoring.

4. Cryptography and C2 Communication

The binary incorporates a sophisticated hybrid cryptographic suite:

Symmetric Ciphers: AES-0.8.4 (likely AES-GCM based on "aes-gcm-key" string), ChaCha20-0.9.1 (stream cipher, common in modern malware)

Authentication: Poly1305-0.8.0 (MAC), HMAC-0.12.1

Key Exchange: curve25519-dalek-4.1.3 (Curve25519 ECDH)

Key Derivation: HKDF-0.12.4, Argon2-0.5.3 (memory-hard KDF, password hashing)

Post-Quantum Cryptography: "mlkem-placeholder-v1" indicates ML-KEM (CRYSTALS-Kyber) preparation — a post-quantum key encapsulation mechanism

Custom Encryption Labels:

- "chameleon-aes-v1" — custom AES variant implementation
- "chameleon-hybrid-v1" — hybrid encryption scheme (likely AES + Curve25519)
- "session-key" — session encryption key management

This cryptographic arsenal enables:

- Encrypted C2 communication resistant to traffic analysis
- Secure key exchange over untrusted networks
- Potential for encrypted payload storage/transmission
- Password/cookie decryption from browser stores (Argon2)

5. Browser Data Theft

The binary targets three major browsers' credential stores:

- \Google\Chrome\User Data (contains Login Data, Cookies, Web Data SQLite databases)
- \Mozilla\Firefox\Profiles (contains logins.json, key4.db, cert9.db)
- \Microsoft\Edge\User Data (Chromium-based, same data structure as Chrome)

The decompression/decryption capabilities (AES-GCM, ChaCha20) would enable Chrome/Edge cookie and password decryption using the browser's stored encryption keys.

6. Payload Architecture

92%+ of the binary consists of a single .rdata section (20.8 MB). This likely contains:

- Encrypted/compressed payload for injection
- Embedded JPEG resources (confirmed by JPEG magic byte detection)
- Configuration data (target URLs, C2 endpoints)
- Browser data parsers and data format handlers

The high-entropy .tls section suggests TLS callbacks execute anti-debug checks before the main code runs, potentially decrypting the payload or modifying execution flow.

Proof of Concept

Static Analysis Reproduction Steps

The following analysis was performed entirely through static reverse engineering — the binary was never executed.

Step 1: File Identification

```
# Hash verification
sha256sum javaw.exe
# Result: 6df297c6ee6c5278dbee4c1c553e71dfd4acae9b559012c22b8e836e21dfdfa3

# PE analysis
python3 -c "
import pefile
pe = pefile.PE('javaw.exe')
print(f'Machine: {hex(pe.FILE_HEADER.Machine)}')
print(f'Sections: {pe.FILE_HEADER.NumberOfSections}')
print(f'Subsystem: {pe.OPTIONAL_HEADER.Subsystem}')
print(f'Entry point: {hex(pe.OPTIONAL_HEADER.AddressOfEntryPoint)}')
"
```

Step 2: Section Entropy Analysis

Calculate entropy per section to identify packed/encrypted regions:

```
import pefile, math
pe = pefile.PE('javaw.exe')
for sec in pe.sections:
    data = sec.get_data()
    entropy = sum(-(data.count(b)/len(data)) *
math.log2(data.count(b)/len(data)) if data.count(b) > 0 else 0 for b in
range(256))
    print(f'{sec.Name.decode().strip():8} {entropy:.4f}')
```

Step 3: String Extraction and IOC Harvesting

```
strings -n 8 javaw.exe | grep -i -E
'(chrome|firefox|edge|sandbox|vmware|vbox|ntopen|ntwrite|ntcreate|aes|gcm|chac
ha|argon|curve|mlkem|chameleon)'
```

Step 4: Import/Export Analysis

```
pe = pefile.PE('javaw.exe')
for entry in pe.DIRECTORY_ENTRY_IMPORT:
    print(f'DLL: {entry.dll.decode()}')
    for imp in entry.imports:
        if imp.name: print(f' {imp.name.decode()}')
```

Step 5: Embedded Content Scanning

Scan for embedded file signatures in large sections:

```
# Check for embedded JPEGs, PEs, archives in .rdata section
rdata_data = pe.sections[2].get_data()
for i in range(len(rdata_data)):
    if rdata_data[i:i+2] == b'\xff\xd8': # JPEG SOI marker
        print(f'JPEG found at offset {hex(i)}')
```

Key Findings

1. Entry point in .data section (not .text) — anomalous
2. .tls section entropy 7.53 (packed/encrypted)
3. .rdata section 20.8 MB (98.6% of binary)
4. Direct NT syscall strings present for process injection
5. Chrome/Firefox/Edge browser data paths embedded
6. 40+ sandbox/VM detection indicators
7. AES-GCM, ChaCha20, Argon2, Curve25519 crypto libraries
8. ML-KEM (post-quantum) placeholder present
9. "chameleon" custom encryption scheme labels

```
#!/usr/bin/env python3
"""
Static Malware Analysis Tool - javaw.exe
Extracts IOCs, analyzes PE structure, detects malicious capabilities.
WARNING: Do NOT execute the analyzed file.
"""

import pefile
import math
import hashlib
import struct
import sys
import os

def analyze_malware(filepath: str) -> dict:
    """Perform comprehensive static analysis on a suspicious PE file."""

    if not os.path.exists(filepath):
        print(f"Error: File {filepath} not found")
        return {}

    results = {
        'file_info': {},
        'sections': [],
```

```

    'imports': [],
    'iocs': {
        'browser_targets': [],
        'sandbox_checks': [],
        'nt_syscalls': [],
        'crypto_libs': [],
        'suspicious_strings': []
    }
}

with open(filepath, 'rb') as f:
    data = f.read()

# File hashes
results['file_info']['md5'] = hashlib.md5(data).hexdigest()
results['file_info']['sha1'] = hashlib.sha1(data).hexdigest()
results['file_info']['sha256'] = hashlib.sha256(data).hexdigest()
results['file_info']['size'] = len(data)

# PE parsing
try:
    pe = pefile.PE(data=data)
except Exception as e:
    print(f"PE parse error: {e}")
    return results

results['file_info']['machine'] =
pefile.MACHINE_TYPE.get(pe.FILE_HEADER.Machine, 'Unknown')
results['file_info']['sections_count'] = pe.FILE_HEADER.NumberOfSections
results['file_info']['subsystem'] =
pefile.SUBSYSTEM_TYPE.get(pe.OPTIONAL_HEADER.Subsystem, 'Unknown')
results['file_info']['entry_point'] =
hex(pe.OPTIONAL_HEADER.AddressOfEntryPoint)

# Section analysis with entropy
for sec in pe.sections:
    name = sec.Name.decode('utf-8', errors='replace').strip(chr(0))
    sec_data = sec.get_data() if sec.SizeOfRawData > 0 else b''
    entropy = 0.0
    if sec_data:
        for x in range(256):
            p_x = sec_data.count(x) / len(sec_data)
            if p_x > 0:
                entropy += -p_x * math.log2(p_x)

    section_info = {

```

```

        'name': name,
        'virtual_address': hex(sec.VirtualAddress),
        'virtual_size': hex(sec.Misc_VirtualSize),
        'raw_size': hex(sec.SizeOfRawData) if sec.SizeOfRawData > 0 else
'0',

        'entropy': round(entropy, 4),
        'executable': bool(sec.Characteristics & 0x20000000),
        'writable': bool(sec.Characteristics & 0x80000000),
    }
    results['sections'].append(section_info)

# Import analysis
try:
    if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
        for entry in pe.DIRECTORY_ENTRY_IMPORT:
            dll_name = entry.dll.decode('utf-8', errors='replace')
            for imp in entry.imports:
                if imp.name:
                    results['imports'].append(f"{dll_name}:
{imp.name.decode()}")
except:
    pass

# IOC extraction - scan for known malicious strings
strings_pool = data.decode('utf-8', errors='replace').split('\n')

# Browser targets
browser_patterns = [
    r'\\Google\\Chrome\\User Data',
    r'\\Mozilla\\Firefox\\Profiles',
    r'\\Microsoft\\Edge\\User Data'
]
for pattern in browser_patterns:
    import re
    if re.search(pattern, data.decode('latin-1')):
        results['iocs']['browser_targets'].append(pattern.replace('\\\\',
'\\'))

# Sandbox detection indicators
sandbox_indicators = [
    'vmtoolsd.exe', 'VBoxService.exe', 'vmhgfs.sys', 'VBoxMouse.sys',
    'sbiedll.dll', 'cuckoomon.dll', 'cmdvrt32.dll',
    'C:\\\\analysis', 'C:\\\\sandbox', 'C:\\\\cuckoo',
]
for indicator in sandbox_indicators:
    if indicator.lower() in data.decode('latin-1').lower():

```

```

        results['iocs']['sandbox_checks'].append(indicator)

# NT Syscall detection
nt_syscalls = ['NtOpenProcess', 'NtCreateThreadEx',
'NtWriteVirtualMemory',
                'NtAllocateVirtualMemory', 'NtProtectVirtualMemory',
                'NtMapViewOfSection', 'NtUnmapViewOfSection']
for nt in nt_syscalls:
    for s in strings_pool:
        if nt in s:
            results['iocs']['nt_syscalls'].append(nt)
            break

# Crypto library detection
crypto_libs = ['aes-', 'chacha20-', 'poly1305-', 'argon2-', 'sha2-',
                'hmac-', 'hkdf-', 'curve25519']
for cl in crypto_libs:
    if cl in data.decode('latin-1').lower():
        results['iocs']['crypto_libs'].append(cl)

# Additional suspicious indicators
suspicious = ['chameleon', 'mlkem', 'session-key',
'NtQuerySystemInformation']
for s in suspicious:
    if s in data.decode('latin-1').lower():
        results['iocs']['suspicious_strings'].append(s)

return results

def print_report(results: dict):
    """Print formatted analysis report."""
    print("=" * 60)
    print("MALWARE STATIC ANALYSIS REPORT")
    print("=" * 60)

    fi = results['file_info']
    print(f"\n[FILE INFO]")
    print(f" MD5:    {fi.get('md5', 'N/A')}")
    print(f" SHA1:   {fi.get('sha1', 'N/A')}")
    print(f" SHA256: {fi.get('sha256', 'N/A')}")
    print(f" Size:   {fi.get('size', 0)} bytes")
    print(f" Machine: {fi.get('machine', 'N/A')}")
    print(f" Sections: {fi.get('sections_count', 0)}")
    print(f" Subsystem: {fi.get('subsystem', 'N/A')}")
    print(f" Entry Point: {fi.get('entry_point', 'N/A')}")

```

```

print(f"\n[SECTION ANALYSIS]")
for sec in results['sections']:
    entropy_flag = ' *** HIGH ***' if sec['entropy'] > 7.0 else (' **' if
sec['entropy'] > 6.2 else '')
    perms = ''
    if sec['executable']: perms += 'X'
    if sec['writable']: perms += 'W'
    print(f" {sec['name']:<10} {sec['raw_size']:<10} ent=
{sec['entropy']:.4f} [{perms}]{entropy_flag}")

print(f"\n[IOCS - Browser Data Theft]")
for bt in results['iocs']['browser_targets']:
    print(f" {bt}")

print(f"\n[IOCS - Sandbox/VM Detection]")
for sc in results['iocs']['sandbox_checks']:
    print(f" {sc}")

print(f"\n[IOCS - NT Syscall Injection]")
for ns in results['iocs']['nt_syscalls']:
    print(f" {ns}")

print(f"\n[IOCS - Cryptography]")
for cl in results['iocs']['crypto_libs']:
    print(f" {cl}")

print(f"\n[IOCS - Suspicious Strings]")
for ss in results['iocs']['suspicious_strings']:
    print(f" {ss}")

if __name__ == '__main__':
    target = sys.argv[1] if len(sys.argv) > 1 else 'javaw.exe'
    if not os.path.exists(target):
        print(f"File not found: {target}")
        sys.exit(1)
    results = analyze_malware(target)
    print_report(results)

```

Remediation

Given this is a malicious software sample (not a vulnerability in a legitimate application), the remediation approach differs:

1. **Detection Rules:** Create YARA rules to detect this binary in transit and at rest:

- Rule targeting the compound "chameleon-aes-v1aes-gcm-key" string
- Rule targeting the compound NT syscall string pattern
- Rule detecting the high-entropy .tls section with raw_size >> virtual_size
- Rule for the specific SHA256 hash

2. **Network Detection:** Deploy network signatures for:

- Chacha20/AES-GCM encrypted traffic patterns
- Curve25519 key exchange handshake patterns
- Connections to unknown/unusual destinations from processes named javaw.exe

3. **Endpoint Detection:**

- Monitor for javaw.exe launching from non-standard locations (not from C:\Program Files\Java)
- Alert on javaw.exe performing direct syscalls (NtOpenProcess, NtCreateThreadEx)
- Detect javaw.exe accessing browser profile directories (Chrome/Firefox/Edge User Data)
- Monitor for Timing API abuse (QueryPerformanceCounter in rapid succession)

4. **Prevention:**

- Block execution of unsigned javaw.exe binaries from non-Java directories
- Restrict browser credential store access to only legitimate browser processes
- Deploy application control policies blocking unsigned binaries in user-writable directories

5. **Hunting Queries:**

- Search for processes named javaw.exe with parent processes other than java.exe/javaw.exe
- Look for javaw.exe making direct syscalls (indicative of userland hook bypass)
- Search for `chameleon-aes-v1` or `chameleon-hybrid-v1` in process memory dumps

6. **YARA Rule:**

```
rule javaw_exe_infostealer {
  meta:
    description = "Detects javaw.exe Rust-based InfoStealer with process injection"
    author = "Security Analysis"
    hash =
"6df297c6ee6c5278dbee4c1c553e71dfd4acae9b559012c22b8e836e21dfdfa3"
    strings:
      $chameleon = "chameleon-aes-v1" nocase
      $nt_syscalls = "NtOpenProcessNtCreateThreadExNtWriteVirtualMemory"
      $browser_chrome = "\\Google\\Chrome\\User Data"
      $browser_firefox = "\\Mozilla\\Firefox\\Profiles"
      $browser_edge = "\\Microsoft\\Edge\\User Data"
```

```
$sandbox_path = "C:\\\\sandbox" nocase
$sandbox_cuckoo = "C:\\\\cuckoo" nocase
$mlkem = "mlkem-placeholder-v1" nocase
$session_key = "session-key"
condition:
  ($chameleon or $nt_syscalls or $mlkem) and
  ($browser_chrome or $browser_firefox or $browser_edge)
}
```